C9 Governance and security

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track C: Microsoft Fabric 2

C9 Governance and security 3

C9-L1 Domains and endorsement, finding the right data 3

C9-L2 OneLake security, roles down to rows and columns..... 7

C9-L3 Sensitivity labels, Purview and lineage 12

C9-L4 Audit evidence, Purview scans, label coverage and the audit log..... 16

C9-L5 Attribute-based access, rows by business unit 21

Track C best-practice checklist 26

Appendix: solutions and answer keys 27

Track C: Microsoft Fabric

Unify data, analytics and Copilot in Fabric, and verify what Copilot produces.

C9 Governance and security

Roadmap: Months 16 to 18: MLOps and deployment; Current work. Domains, OneLake security, sensitivity labels, Purview and lineage.

C9-L1 Domains and endorsement, finding the right data

Concept

A tenant with 400 workspaces needs two things before anyone trusts its data: a way to find data by business area, and a way to tell the authoritative copy from the rest. Fabric gives you **domains** and **endorsement**. Microsoft Learn shows neither with a preview label (checked September 2026).

Domains group workspaces by business area, such as Finance or Marketing. You can split a domain into **subdomains**.

- You associate **workspaces** with a domain. Every item in the workspace inherits the domain.
- Users filter the **OneLake catalog** by domain to find relevant data.
- Some tenant settings can be **delegated** to domain level, so each area sets its own rules.
- **Domain assignment does not grant or restrict access.** Workspace roles and item permissions decide who sees what.

Three roles manage domains. **Fabric admins** create domains and name domain admins. **Domain admins**, ideally the business owners, edit their domain and associate workspaces. **Domain contributors** are workspace admins allowed to assign their own workspaces to the domain.

Endorsement labels an item with one of three badges:

Badge	Meaning	Who applies it
Promoted	The creators think it is ready to share	Anyone with write access to the item
Certified	An authorised reviewer confirms it meets the organisation's standards	Only users a Fabric admin names
Master data	The authoritative source for core data such as customer lists	Only users a Fabric admin names

Any item except a Power BI dashboard can be promoted or certified. Master data applies only to items that hold data, such as lakehouses and semantic

models. Certification and master data work only after a Fabric admin enables them.

Endorsed items get a badge and appear first in some lists. That helps people pick the right copy. It does not stop anyone using an uncertified one.

Why it matters

A company had five semantic models called "Sales" across four workspaces. Two finance directors presented different quarterly revenue to the board. Both had used a model called "Sales". Certifying one model, retiring the others, and filing the finance workspaces under a Finance domain made the right model the first result anyone found.

Worked example: weak code

```
1  Workspaces: 400, no domains
1  Semantic models named "Sales": 5, none endorsed
2  Access plan: put finance workspaces in the Finance domain to restrict them
```

1. **Line 2.** Nothing tells users which copy is authoritative.
2. **Line 3.** Domains do not control access. Workspace roles do.

Worked example: strong code

```
1  Domains: Finance, Marketing, Operations (subdomain Finance > Tax)
2  Domain admins: business owners. Domain contributors: finance workspace
     admins
1  Certified: finance_sales_model (by a Fabric-admin-approved certifier)
4  Master data: crm_lakehouse, the single home of dim_customer
2  Access: workspace roles and OneLake security (lesson C9-L2)
```

1. **Line 3.** One authoritative model, labelled by an authorised reviewer.
2. **Line 5.** Discovery and access are separate controls.

Domains help people find data. Endorsement tells them which copy to trust. Neither controls access.

Recap card

- **What a domain does** Groups workspaces by business area for discovery and delegated settings. Does not control access.
- **Domain roles** Fabric admin creates domains. Domain admin edits one. Domain contributor, a workspace admin, assigns their workspaces.
- **Endorsement badges** Promoted (creators), Certified (named reviewers), Master data (named users, data items only). Dashboards get none.

- **Does endorsement block use of other copies?** No. It adds a badge and ranks the item first in some lists.

Practice

Exercise 1. Multiple choice (level 1)

You associate a workspace with the Finance domain. What changes for users outside finance?

- a) They lose access to the workspace
- b) Nothing about access. The workspace's items show under Finance in the OneLake catalog
- c) They gain read access to the items
- d) They must request access from the domain admin

Why?

- r1) Domain assignment does not affect item visibility or access. Workspace roles and item permissions do.
- r2) Domains only apply to Power BI items.

Exercise 2. Multiple choice (level 2)

A data engineer with Contributor access to a lakehouse wants to mark it as **Certified**. What happens?

- a) It works: Contributors can certify their items
- b) They can request certification. Only users a Fabric admin specifies can certify
- c) Certification is automatic once a data quality test passes
- d) Lakehouses cannot be certified

Why?

- r1) Any user can request certification, but only users specified by a Fabric administrator can certify.
- r2) Only workspace Admins can certify.

Exercise 3. Fill the gap (level 2)

Complete the domain roles.

```
1 Creates domains and names domain admins: Fabric _____(1)
2 Assigns their own workspace to a domain: domain _____(2), who must be a
    workspace admin
```

Exercise 4. Write the function (level 3)

Write `allowed_badges(item_type: str) -> set[str]` for the endorsement rules. Use the badge names `"promoted"`, `"certified"` and `"master_data"`. A `"dashboard"` gets no badges. Data items (`"lakehouse"`, `"warehouse"`, `"semantic_model"`, `"kql_database"`) get all three. Every other item type gets promoted and certified. Keep the data item types in a module constant.

Your code must pass these checks:

- Power BI dashboards cannot be promoted or certified.
- Items that hold data take all three badges.
- Master data applies only to items that hold data.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Module constant.
DATA_ITEM_TYPES = ...

# Module constant.
NO_BADGE_TYPES = ...

# Endorsement badges an item type can receive.
def allowed_badges(item_type: str) -> set[str]: ...
```

Starter code:

```python
def allowed_badges(item_type):
    return {"promoted", "certified", "master_data"}
```

Exercise 5. Architecture design (level 4)

Organise a tenant where finance has 30 workspaces, three copies of the customer table, and complaints that nobody finds the right sales model.

Options: A Finance domain with finance workspaces associated, finance leads as domain admins, A naming convention only: prefix finance workspaces with fin_.

1. Grouping
2. The customer table
3. Keeping non-finance users out of finance data

Sources

- Workspaces associate with domains and items inherit; OneLake catalog filters by domain; tenant settings can be delegated; domain assignment doesn't affect visibility or access; roles Fabric admin, domain admin, domain contributor (must be workspace admin); subdomains. (verified:

https://learn.microsoft.com/en-us/fabric/governance/domains (updated 2025-05-01))

- Badges Promoted, Certified, Master data; any item except dashboards can be promoted (by users with write permission) or certified; only admin-specified users certify or label master data; master data only for data items; must be enabled by a Fabric admin. (verified: https://learn.microsoft.com/en-us/fabric/governance/endorsement-overview (updated 2024-07-11))

C9-L2 OneLake security, roles down to rows and columns

Concept

OneLake security controls who reads which data in OneLake: tables, folders, schemas, rows and columns. Every supported engine enforces it (Microsoft Learn, checked September 2026).

Roles. Each role has a permission (Read or ReadWrite), the data it covers, and its members: Entra users or groups. The model is **deny by default** and supports only **grant** roles. There are no deny roles. A lakehouse starts with a **DefaultReader** role that gives users with the ReadAll permission access to its data. Remove or narrow it before your own roles mean anything.

Who it restricts. OneLake security governs **Viewers** and users with Read permission on the item. Workspace Admins, Members and Contributors already have data access through their workspace role. Make analysts Viewers.

Row and column security. A role can filter rows (RLS) and hide columns (CLS) on a table. Within one role, access is the **intersection** of its table, row and column rules. Across roles, access is the **union**: row filters combine with OR, so a user in roles for `city = 'Leeds'` and `city = 'York'` sees both cities. Two exceptions:

- In the **SQL analytics endpoint**, CLS uses deny semantics: if any of the user's roles hides a column, the endpoint blocks it.
- `SELECT *` on a table with hidden columns returns the allowed columns in Spark, but **fails** in the SQL analytics endpoint.

Keep row and column rules that belong together in **the same role**. OneLake rejects a combination where two roles allow different columns of a table and either role also filters its rows. Roles with ReadWrite cannot carry RLS or CLS.

Engines. RLS and CLS are GA in Spark notebooks, lakehouses, Direct Lake on OneLake semantic models, and the SQL analytics endpoint in **user's identity** access mode. Eventhouse supports RLS only, in preview.

Why it matters

A team added a OneLake security role limiting analysts to UK rows of `fact_sales`, then found the analysts still saw every country. Two causes. The analysts were Contributors on the workspace, so roles did not restrict them. And the DefaultReader role was still in place, granting full reads to every Viewer with ReadAll. Making analysts Viewers and removing DefaultReader fixed both.

Worked example: weak code

```
1 Analysts' workspace role: Contributor
2 Roles: DefaultReader (unchanged), uk_sales (fact_sales where country =
    'UK')
3 Salary hiding: role hr_cols hides salary, role hr_rows filters team =
    'Ops'
```

1. **Line 1.** Contributors already have data access. OneLake roles do not restrict them.
2. **Line 2.** DefaultReader still grants full reads. The union of roles gives everything.
3. **Line 3.** Row and column rules split across roles. Keep them in one role.

Worked example: strong code

```
1 Analysts' workspace role: Viewer
  Roles: DefaultReader removed. uk_sales (fact_sales where country = 'UK')
2 Salary hiding: role hr_ops (dim_employee where team = 'Ops', columns
    without salary)
3 SQL analytics endpoint: user's identity access mode
```

1. **Line 1.** Viewers are the users OneLake security governs.
2. **Line 3.** Rows and columns in one role: the intersection applies.
3. **Line 4.** The endpoint enforces OneLake security only in this mode.

Readers are Viewers, defaults are removed, related row and column rules share a role, and every engine runs in a mode that enforces them.

Recap card

- **OneLake security combines roles how?** Within a role: intersection of table, row and column rules. Across roles: union. Row filters OR together.
- **Whom does OneLake security restrict?** Viewers and Read-permission users. Admins, Members and Contributors already have data access.

- **SELECT * with a hidden column** Spark returns allowed columns. SQL analytics endpoint returns an error.
- **Engines enforcing OneLake RLS and CLS (GA)** Spark, lakehouse, Direct Lake on OneLake, SQL analytics endpoint in user's identity mode. Eventhouse: RLS only, preview.

Practice

Exercise 1. Multiple choice (level 1)

A user belongs to two OneLake security roles on `fact_sales`: one filters `region = 'North'`, the other `region = 'South'`. Which rows do they see?

- a) North and South rows
- b) No rows: the filters contradict each other
- c) North rows only: the first role wins
- d) All rows

Why?

- r1) Roles combine as a union. Row filters from different roles combine with OR.
- r2) OneLake picks the most permissive single role.

Exercise 2. Multiple choice (level 2)

An analyst is a **Contributor** on the workspace. A OneLake security role limits them to UK rows. What do they see in Spark?

- a) UK rows only
- b) All rows: Contributors already have data access through their workspace role
- c) No rows
- d) An error

Why?

- r1) OneLake security governs Viewers and users with Read permission. Admins, Members and Contributors already have data access.
- r2) Spark ignores OneLake security.

Exercise 3. Multiple choice (level 2)

A table hides the `salary` column from a user. They run `SELECT * FROM dim_employee` in Spark, then in the SQL analytics endpoint. What happens?

- a) Spark returns the allowed columns. The endpoint returns an error

- b) Both return the allowed columns
- c) Both return an error
- d) Both return salary as null

Why?

- r1) Microsoft Learn documents that CLS behaves differently: Spark shows only allowed columns, the endpoint blocks the query.
- r2) Spark does not support column security.

Exercise 4. Write the function (level 3)

Write `visible_columns(roles: list[dict[str, set[str]]], table: str, engine: Literal["spark", "sql_endpoint"]) -> set[str]`. Each role maps table names to the columns it allows. Consider only roles that include `table`. If none do, return an empty set (deny by default). For `"spark"`, return the union of the allowed columns. For `"sql_endpoint"`, return the intersection, because a column hidden by any role is blocked. Raise `ValueError` for any other engine.

What you need to know.

`Literal` from `typing` limits a parameter to fixed values: `mode: Literal["fast", "safe"]` tells readers and mypy that only those two strings are valid. Python does not check it at run time, so a caller can still pass any string. Raise an error yourself when a wrong value must not get through.

`set().union(*sets)` combines sets: every item in any of them. `set.intersection(*sets)` keeps only items in all of them.

Your code must pass these checks:

- Spark combines column access across roles as a union.
- The SQL analytics endpoint blocks a column hidden by any role: intersect.
- Ignore roles that do not include the table. No role for the table means no access.
- Unknown engines raise ValueError.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Columns a user sees in one table across their OneLake security roles.
def visible_columns(
    roles: list[dict[str, set[str]]],
    table: str,
    engine: Literal['spark', 'sql_endpoint'],
) -> set[str]: ...
```

Starter code:

```python
def visible_columns(roles, table, engine):
    columns = set()
    for role in roles:
        columns |= role[table]
    return columns
```

Exercise 5. Architecture design (level 4)

HR analysts must see employee rows for their own region, without the `salary` column, in Spark and through a Direct Lake report.

Options: Viewer, Contributor.

1. Analysts' workspace role
2. Role design
3. Semantic model mode
4. DefaultReader role

Sources

- Role components (Read/ReadWrite, data, members); deny by default; grant roles only; DefaultReader role; default roles and ReadWrite matter only for Viewers since Admin/Member/Contributor have write access; ReadWrite roles cannot contain RLS/CLS. (verified: https://learn.microsoft.com/en-us/fabric/onelake/security/data-access-control-model (updated 2026-08-28); get-started-security (updated 2026-09-01))
- Within a role OLS∩RLS∩CLS; across roles union; RLS combines with OR; SQL analytics endpoint intersects CLS; roles with different columns plus RLS on the same table unsupported. (verified: https://learn.microsoft.com/en-us/fabric/onelake/security/data-access-control-model (updated 2026-08-28))
- SELECT * with hidden columns: Spark shows allowed columns, SQL analytics endpoint returns an error. (verified: https://learn.microsoft.com/en-us/fabric/onelake/security/table-column-row-security (updated 2026-08-19))
- RLS/CLS GA in lakehouse, Direct Lake on OneLake, Spark notebooks, SQL analytics endpoint in user's identity mode; Eventhouse RLS only in public preview; third-party engines preview. (verified: https://learn.microsoft.com/en-us/fabric/onelake/security/read-secured-data (updated 2026-08-28))

C9-L3 Sensitivity labels, Purview and lineage

Concept

Access control decides who opens an item. **Sensitivity labels** say how sensitive its content is, and travel with it. Fabric uses labels from **Microsoft Purview Information Protection**. You need a Purview licence, and labelling Power BI items also needs a Pro or Premium Per User licence (Microsoft Learn, checked September 2026).

How labels spread:

- **Downstream inheritance.** A label on a semantic model flows to the reports and dashboards built on it. It never overwrites a label someone applied manually, and never replaces a label with a less restrictive one.
- **Default and mandatory labelling.** Admins can apply a default label to new items, or require users to pick one before they save.
- **Export.** Labels, and the protection a Purview policy attaches, stay with data exported to Excel, PDF and PowerPoint.

Protection policies link a label to access. When an item carries the label, users and groups named in the policy keep the permissions they already have. Everyone else is blocked. The user who last applied the label is not blocked.

Purview also gives you:

- **Unified Catalog**: metadata about Fabric items, visible in Purview.
- **Data loss prevention (DLP)**: detects sensitive data uploaded to OneLake and raises policy tips and alerts. It also restricts access to sensitive assets in KQL databases, SQL databases and warehouses.
- **Audit**: compliance admins track label activity.

Lineage shows how items connect. Every workspace has a lineage view that any user with a workspace role can open. It covers all items in the workspace, plus external sources one step upstream. **Impact analysis** starts from one item and lists the downstream items and workspaces a change might affect, and lets you notify their contacts.

Run impact analysis before a schema change. A label names the risk. A protection policy acts on it.

Why it matters

An engineer renamed a column in a lakehouse table on a Friday. Monday morning, 14 reports in three other workspaces showed errors, and a finance close was delayed. Impact analysis on the table would have listed all 14 reports and their workspaces, and notified their owners before the change.

Worked example: weak code

```
  1 Change: rename cust_id to customer_id in fact_sales
(1) Checks: none, tested in the dev lakehouse notebook
(2) Label on hr_salaries model: none, "the workspace is private"
```

1. **Line 2.** Your notebook works. The 14 downstream reports break.
2. **Line 3.** Exports to Excel leave the workspace. A label and policy would travel with them.

Worked example: strong code

```
  1 Change: rename cust_id to customer_id in fact_sales
(1) Checks: impact analysis lists 14 reports in 3 workspaces. Contacts
      notified. Views keep cust_id for one release
(2) Label on hr_salaries model: Highly Confidential, with a protection policy
      for the HR group
(3) Downstream: reports inherit Highly Confidential. Excel exports keep the
      label
```

1. **Line 2.** Know who breaks, tell them, and give them time.
2. **Line 3.** The policy blocks anyone outside the HR group, even if someone shares the item by mistake.
3. **Line 4.** Protection follows the data out of Fabric.

Run impact analysis before changes. Label sensitive items, attach protection policies, and let inheritance carry the label downstream.

Recap card

- **Downstream label inheritance rules** Flows to downstream items. Never overwrites a manual label. Never downgrades.
- **Protection policy** Tied to a label. Named users and groups keep existing permissions. Everyone else is blocked.
- **Lineage view** Every workspace has one. All items in the workspace plus external sources one step upstream.
- **Impact analysis** Downstream items and workspaces a change might affect, with a way to notify their contacts.

Practice

Exercise 1. Multiple choice (level 1)

A report built on a **Confidential** semantic model has a **Highly Confidential** label that someone applied by hand. The model's label changes to **General**. What happens to the report's label?

- a) It becomes General
- b) It stays Highly Confidential
- c) It becomes Confidential
- d) It is removed

Why?

- r1) Downstream inheritance never overwrites a manually applied label, and never downgrades a label.
- r2) Reports never inherit labels.

Exercise 2. Multiple choice (level 2)

You plan to change a column type in a lakehouse table. Which tool lists every downstream item and workspace that the change might break?

- a) Impact analysis
- b) The OneLake catalog domain filter
- c) Sensitivity labels
- d) The Capacity Metrics app

Why?

- r1) Impact analysis shows affected downstream items and workspaces and can notify their contacts.
- r2) Impact analysis blocks the change until owners approve.

Exercise 3. Fill the gap (level 2)

Complete the protection policy behaviour.

```
Item labelled Highly Confidential, policy allows the HR group.
HR group member with Viewer access: _____(1) their Viewer access.
Finance user with Viewer access: _____(2).
```

Exercise 4. Write the function (level 3)

Write `inherit_label(current: str | None, manual: bool, upstream: str, order: list[str]) -> str | None`. It returns the downstream item's label after its upstream item's label becomes `upstream`. `order` lists labels from least to

most restrictive. Rules: a manual label is kept. An unlabelled item takes `upstream`. Otherwise the item takes `upstream` only if it is more restrictive than `current`. Raise `ValueError` if a label is not in `order`.

Your code must pass these checks:

- Inheritance fills in and raises labels.
- Inheritance never replaces a label with a less restrictive one.
- Inheritance never overwrites a manually applied label.
- Unknown labels raise ValueError.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Downstream label after the upstream label changes.
def inherit_label(
    current: str | None,
    manual: bool,
    upstream: str,
    order: list[str],
) -> str | None: ...
```

Starter code:

```python
def inherit_label(current, manual, upstream, order):
    return upstream
```

Exercise 5. Architecture design (level 4)

Protect a payroll semantic model and its reports. Analysts outside HR sometimes receive shared links by mistake, and HR exports summaries to Excel.

Options: Highly Confidential, with reports inheriting it downstream, No label, the workspace is restricted.

1. Label on the payroll model
2. Access control for mis-shared links
3. Before renaming a payroll column

Sources

- Labels come from Purview Information Protection; Purview licence plus Pro or PPU needed to label Power BI items; labels persist on export to Excel, PDF, PowerPoint; default and mandatory labelling exist. (verified: https://learn.microsoft.com/en-us/fabric/governance/information-protection (updated 2026-07-13))

- Downstream inheritance never overwrites manual labels or downgrades to a less restrictive label. (verified: https://learn.microsoft.com/en-us/fabric/governance/service-security-sensitivity-label-downstream-inheritance (updated 2026-07-13))
- Protection policies: users in the policy retain existing permissions, others blocked; label issuer exempt. (verified: https://learn.microsoft.com/en-us/fabric/governance/protection-policies-overview (updated 2025-12-03))
- Purview Unified Catalog, DLP on OneLake uploads and access restriction for KQL, SQL database, warehouse; audit of label activity. (verified: https://learn.microsoft.com/en-us/fabric/governance/microsoft-purview-fabric (updated 2026-02-09))
- Every workspace has a lineage view accessible to any workspace role; shows workspace items and external sources one step upstream; impact analysis lists downstream items and workspaces and notifies contacts. (verified: https://learn.microsoft.com/en-us/fabric/governance/lineage (updated 2023-11-06); impact-analysis (updated 2023-05-23))

C9-L4 Audit evidence, Purview scans, label coverage and the audit log

Concept

An auditor asks three questions about talent data. What do you hold? How is it protected? Who touched it? Answer each with evidence from a system, dated, not with a statement of intent. Fabric and Purview give you one source for each.

What you hold: a Purview Data Map scan. Register the Fabric tenant in the Microsoft Purview Data Map and scan it, in full, incrementally, or scoped to chosen workspaces. The scan brings in item metadata and lineage. For lakehouses it also reads tables and files. Two limits matter. The scan does not classify or label Fabric data. And once metadata reaches Purview, Purview permissions, not Fabric roles, decide who sees it, so restrict the collection that holds HR items.

How it is protected: label coverage. The **Govern** tab of the OneLake catalog reports the share of items with no sensitivity label, by workspace and item type. It refreshes once a day, and it does not count tables inside an item.

Who touched it: the audit log. Fabric records activities in the Microsoft Purview **unified audit log**. You need the Audit Logs role to search it. Export results to CSV, or script the search with `Search-UnifiedAuditLog`. **Audit (Standard)** keeps records for 180 days. **Audit (Premium)** keeps them up to a year by policy, longer with an add-on licence.

If your audit period is longer than your retention, export on a schedule into storage you control.

An evidence pack holds the scan date and scope, label coverage per workspace with the gaps listed, and the access records for sensitive items, each with the query that produced it. Status: the scan and the audit log are generally available. The Govern tab has no preview label but is still rolling out by region (Microsoft Learn, November 2026).

Why it matters

An internal audit asked who had opened the salary semantic model in the last financial year. The team ran an audit search in month 11. Audit (Standard) returned only the last six months, because the first five had expired. The finding read "access cannot be evidenced for 5 of 12 months". A monthly export would have closed it.

Worked example: weak code

```
# Evidence pack, HR workspace
inventory: we know what is in there
labels: all sensitive items are labelled
access: run an audit search when the auditor asks
```

1. **Line 2.** A belief, not evidence. Nothing shows when it was checked or what it covered.
2. **Line 3.** No coverage figure and no list of gaps.
3. **Line 4.** Records past the 180-day retention will be gone by then.

Worked example: strong code

```
# Evidence pack, HR workspace, period April 2026 to March 2027
inventory:
  source: Purview Data Map scan, scoped to HR workspaces
  run: 2027-04-02, incremental, 214 items
labels:
  source: OneLake catalog, Govern, label coverage
  coverage: 97% labelled, 6 unlabelled items listed with owners
access:
  source: unified audit log, monthly CSV export to the evidence store
  query: activities on hr_salaries_model, all users, each month
```

```
❹ collection_access: Purview collection restricted to HR data stewards
```

1. **Line 4.** Dated, scoped and counted. An auditor can repeat it.
2. **Line 7.** A figure plus the gaps, each with someone to fix it.
3. **Line 9.** Monthly exports keep a year of records that Audit (Standard) would drop after 180 days.
4. **Line 11.** Scanned metadata follows Purview permissions, so restrict who sees it.

Answer what you hold, how it is protected and who touched it with dated system evidence. Export audit records before they expire, and restrict who sees scanned HR metadata in Purview.

Recap card

- **Three audit questions and their sources** What you hold: a Purview Data Map scan. How it is protected: label coverage in the OneLake catalog. Who touched it: the unified audit log.
- **Audit (Standard) retention** 180 days. Audit (Premium) keeps up to a year by policy, longer with an add-on. Export on a schedule for longer periods.
- **Who sees scanned Fabric metadata in Purview?** Purview permissions decide, not Fabric roles. Restrict the collection that holds HR items.
- **What a Fabric scan does not do** It does not classify or label Fabric data. It brings in metadata and lineage.

Practice

Exercise 1. Multiple choice (level 1)

Which source answers "who opened the salary report last month?"

- a) The unified audit log in Microsoft Purview
- b) A Purview Data Map scan
- c) Label coverage in the OneLake catalog
- d) The lineage view

Exercise 2. Multiple choice (level 2)

Your audit period is 12 months and you have Audit (Standard). What should you do?

- a) Export audit records on a schedule into storage you control
- b) Search the log once, at the end of the period
- c) Nothing, because Audit (Standard) keeps records for a year
- d) Rely on the Data Map scan instead

Why?

- r1) Audit (Standard) keeps records for 180 days, so the first months would expire before the end.
- r2) Exports are needed because the portal cannot search more than seven days.

Exercise 3. Fill the gap (level 2)

Complete the three questions and their sources.

```
1 What do you hold?          a Purview Data Map _____(1) of the Fabric
    tenant
2 How is it protected?       label _____(2) in the OneLake catalog Govern
    tab
3 Who touched it?            the unified _____(3) log
```

Exercise 4. Write the function (level 3)

Write `label_coverage(items: list[dict]) -> dict[str, float]`. Each item is a dict with `"workspace"` and `"label"`, where `"label"` is a label name or `None`.

Return, for each workspace that has items, the share of its items with a label, from 0.0 to 1.0. Return an empty dict for an empty list. Raise `ValueError` if an item has no `"workspace"` key.

What you need to know.

`dict.setdefault(key, default)` returns the value for `key`, adding `default` first if the key is missing:

```
1 counts = {}
2 counts.setdefault("red", [0, 0])[0] += 1
3 counts   # {"red": [1, 0]}
```

`"key" in some_dict` checks whether a key exists: `"a" in {"a": 1}` is `True`.

Your code must pass these checks:

- Return the share of labelled items for each workspace.
- An empty list has no workspaces, so return an empty dict without dividing by zero.
- Raise ValueError when an item has no workspace.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
1 # Share of items with a sensitivity label, per workspace.
2 def label_coverage(items: list[dict]) -> dict[str, float]: ...
```

Starter code:

```
1 def label_coverage(items):
```

```
    labelled = sum(1 for item in items if item["label"])
    return {"all": labelled / len(items)}
```

Exercise 5. Architecture design (level 4)

Design the evidence process for the HR workspaces before the first annual audit.

Options: A scheduled Purview scan scoped to HR workspaces, with its date and item count kept, A spreadsheet the team updates when it remembers.

1. Inventory
2. Access records
3. Who sees scanned HR metadata in Purview

Sources

- A Purview Data Map scan of a same-tenant Fabric tenant supports full, incremental and scoped scans and brings in metadata and lineage. For items other than Power BI, only item-level metadata and lineage, with sub-item metadata for lakehouse tables and files. Once metadata is in Purview, Purview permissions, not Fabric permissions, decide who sees it. (verified: https://learn.microsoft.com/en-us/purview/register-scan-fabric-tenant (ms.date 2026-04-28, updated 2026-07-31))
- The Data Map source table lists Fabric with no automatic classification and no sensitivity labels applied to Data Map assets, with lineage and live view supported. (verified: https://learn.microsoft.com/en-us/purview/data-map-data-sources (2026-07-01))
- The OneLake catalog Govern section reports sensitivity label coverage and unlabelled items, refreshes from admin monitoring storage once a day, and does not include subitems such as tables. It is rolling out by region, with no preview label on the page. (verified: https://learn.microsoft.com/en-us/fabric/governance/onelake-catalog-govern (ms.date 2026-08-26))
- Fabric activities are in the Microsoft Purview audit log; searching it needs the Audit Logs role. Audit (Standard) retains 180 days; Audit (Premium) up to one year by policy and ten years with an add-on licence; Search-UnifiedAuditLog and CSV export are available. (verified: https://learn.microsoft.com/en-us/fabric/admin/track-user-activities (2026-02-12) and https://learn.microsoft.com/en-us/purview/audit-solutions-overview (2026-05-18))

C9-L5 Attribute-based access, rows by business unit

Concept

RBAC and ABAC. Role-based access control (RBAC) grants access by membership: you are in the HR analysts role, so you read the HR tables. Attribute-based access control (ABAC) decides from attributes such as department, country or business unit. Talent data needs ABAC, because the usual rule is "a manager sees their own business unit".

OneLake security is role-based. Roles only grant, and deny is the default. Its row filters are static: "RLS roles don't support dynamic and multitable queries" (Microsoft Learn, page dated 19 August 2026). A role cannot look up who is asking. Two patterns bring attributes in.

Pattern 1: attributes build the groups. A Microsoft Entra dynamic membership group adds every user whose attributes match a rule, such as `user.department -eq "HR"`. Map each group to a OneLake security role with a static row filter for that value. Limits: each member needs a Microsoft Entra ID P1 licence, an item holds 250 roles, and a group change takes about an hour to reach OneLake.

Pattern 2: an entitlements table. In a Warehouse or the SQL analytics endpoint, `CREATE SECURITY POLICY` adds a filter predicate written as an inline table-valued function. The function can join an entitlements table on `USER_NAME()`, so one policy serves every user. It filters T-SQL queries on those items only.

Who writes the attributes? Access is only as safe as the attribute behind it. Microsoft advises you to audit who can write each attribute a dynamic group rule reads, in Entra and in on-premises Active Directory. Treat the entitlements table the same way.

Status: OneLake security RLS, Warehouse RLS and Entra dynamic groups are GA (Microsoft Learn, November 2026).

Why it matters

A talent team built one OneLake role per manager, with the manager's name in each row filter. After a reorganisation, 40 roles needed edits by hand, and 6 were missed for two weeks. Managers who had moved still saw their old teams' salaries. An entitlements table that HR maintains, read by one policy, would have needed one data change per move.

Worked example: weak code

```
1  CREATE FUNCTION Security.fn_unit(@unit AS varchar(50))
2      RETURNS TABLE
3  WITH SCHEMABINDING
4  AS
5      RETURN SELECT 1 AS allowed
(1)    WHERE (@unit = 'Retail' AND USER_NAME() = 'a.khan@example.com')
(2)       OR (@unit = 'Audit' AND USER_NAME() = 'j.lee@example.com');
```

1. **Line 6.** People and units hardcoded in the predicate. Every move means a code change, and personal data sits in code.
2. **Line 7.** The function grows by one line per manager and has to be dropped and recreated with its policy each time.

Worked example: strong code

```
1  CREATE FUNCTION Security.fn_unit(@unit AS varchar(50))
2      RETURNS TABLE
3  WITH SCHEMABINDING
4  AS
5      RETURN SELECT 1 AS allowed
(1)    FROM Security.entitlements AS e
(2)    WHERE e.upn = USER_NAME() AND e.business_unit = @unit;
8  GO
9
10 CREATE SECURITY POLICY Security.unit_filter
(3) ADD FILTER PREDICATE Security.fn_unit(business_unit)
12 ON talent.fact_headcount
13 WITH (STATE = ON);
```

1. **Line 6.** Who sees which unit is data, maintained by HR data stewards, not code.
2. **Line 7.** Dynamic: the predicate looks up the person asking. No entitlement row means no rows, deny by default.
3. **Line 11.** One policy on the table serves every user.

OneLake security row filters are static. For "each manager sees their own unit", build groups from Entra attributes or join an entitlements table in a Warehouse security policy. Audit who can write the attributes and the table.

Recap card

- **RBAC and ABAC** RBAC grants by role membership. ABAC decides from attributes such as department or business unit.
- **Can a OneLake security row filter depend on the current user?** No. OneLake RLS roles don't support dynamic or multitable queries. Use Entra dynamic groups mapped to roles, or Warehouse RLS.

- **Dynamic row security in a Fabric Warehouse** `CREATE SECURITY POLICY` with a filter predicate: an inline table-valued function that joins an entitlements table on `USER_NAME()`.
- **Entra dynamic groups for access: limits and risks** Each member needs Entra ID P1. A group change takes about an hour to reach OneLake. Audit who can write the attributes the rule reads.

Practice

Exercise 1. Multiple choice (level 1)

You want one OneLake security role whose row filter shows each user the rows for their own business unit. Why does this not work?

- a) OneLake RLS roles don't support dynamic queries, so a filter cannot look up who is asking
- b) OneLake security has no row-level security
- c) Roles can contain only users, not groups
- d) OneLake security only supports deny roles

Exercise 2. Multiple choice (level 2)

You plan Entra dynamic groups with the rule `user.department -eq "HR"` to grant access to salary rows. What do you check first?

- a) Who can write the department attribute, in Entra and in on-premises Active Directory
- b) Whether the group has a display name
- c) Whether the rule uses the rule builder or the text box
- d) Nothing, because Entra controls dynamic groups

Why?

- r1) Membership is only as safe as the write controls on the attribute the rule reads.
- r2) Dynamic membership groups are in preview.

Exercise 3. Fill the gap (level 2)

Complete the two patterns.

```
1 Entra dynamic group rule:     user.department _____(1) "HR"
2 Warehouse row security:       CREATE _____(2) POLICY ... ADD FILTER
     PREDICATE ...
3 Current user in the predicate: _____(3)()
```

Exercise 4. Write the function (level 3)

Write `allowed_rows(rows: list[dict], entitlements: dict[str, list[str]], user: str) -> list[dict]`. It mirrors an entitlements predicate.

Each row has a `"business_unit"` key. `entitlements` maps a user principal name, in lower case, to the business units that user may see. Return the rows whose business unit the user may see, in their original order.

- Match `user` without regard to case, because user principal names are case-insensitive.
- A user with no entry sees no rows.
- A row with no `"business_unit"` key is never returned.

What you need to know.

`dict.get(key, default)` returns the value for `key`, or `default` when the key is missing:

```
prices = {"tea": 2}
prices.get("tea", 0)      # 2
prices.get("coffee", 0)   # 0
```

`str.lower()` returns a lower-case copy: `"Ann@Example.com".lower()` is `"ann@example.com"`.

Your code must pass these checks:

- Return only the rows for the user's business units, in their original order.
- A user with no entitlement sees no rows. Deny by default.
- Match the user principal name without regard to case.
- Skip a row with no business unit instead of failing or returning it.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Rows the user may see, from an entitlements map.
def allowed_rows(
    rows: list[dict],
    entitlements: dict[str, list[str]],
    user: str,
) -> list[dict]: ...
```

Starter code:

```
def allowed_rows(rows, entitlements, user):
    return rows
```

Exercise 5. Architecture design (level 4)

Design row access for talent reports across 40 business units. Managers see their own unit. HR business partners see the units they support.

Options: An entitlements table keyed on user principal name, maintained by HR data stewards, read by one Warehouse security policy, Names and units written into the predicate function.

1. Where the mapping lives
2. A user with no entitlement
3. Who may change the mapping

Sources

- OneLake security RLS roles don't support dynamic and multitable queries; OneLake security uses grant roles only and is deny by default. (verified: https://learn.microsoft.com/en-us/fabric/onelake/security/table-column-row-security (ms.date 2026-08-19); data-access-control-model (ms.date 2026-08-28))
- OneLake security allows 250 roles per item (1,000 on request) and 500 members per role; role definition changes take about 5 minutes and user group changes about an hour, with some engines needing an extra hour. (verified: https://learn.microsoft.com/en-us/fabric/onelake/security/data-access-control-model (ms.date 2026-08-28, updated 2026-10-16))
- Warehouse and SQL analytics endpoint RLS uses CREATE SECURITY POLICY with inline table-valued function predicates; a predicate can join another table; the documented example compares a column with USER_NAME(); Direct Lake on a warehouse falls back to DirectQuery under RLS. (verified: https://learn.microsoft.com/en-us/fabric/data-warehouse/row-level-security (updated 2026-09-02))
- Entra dynamic membership group rules such as user.department -eq "Sales"; each unique member needs a Microsoft Entra ID P1 licence; Microsoft advises auditing write permissions on attributes used in rules, including those synced from on-premises Active Directory. (verified: https://learn.microsoft.com/en-us/entra/identity/users/groups-dynamic-membership (ms.date 2026-08-13))

Track C best-practice checklist

☐ Check the status (GA or preview) of every Fabric feature before you depend on it.

☐ One copy of each dataset. Use shortcuts or mirroring before you copy.

☐ Lakehouse for Spark and files. Warehouse for T-SQL writes and multi-table transactions.

☐ The SQL analytics endpoint is read-only for table data. Write through Spark or a Warehouse.

☐ Semantic models use star schemas. DAX measures state their filter context.

☐ Direct Lake models are checked for fallback to DirectQuery.

☐ Every Copilot output is read, run on known data and compared against a second method.

☐ Workspaces map to domains. Access uses roles and OneLake security, not item sharing.

☐ Items live in Git. Changes move through deployment pipelines.

☐ Capacity use is monitored, and heavy jobs are scheduled off peak.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

C9-L1 Domains and endorsement, finding the right data

C9-L1 exercise 1

Hints. (1) What are domains for? (2) Finding data. (3) Access stays with roles and permissions.

Answer. b) Nothing about access. The workspace's items show under Finance in the OneLake catalog Because: r1) Domain assignment does not affect item visibility or access. Workspace roles and item permissions do.

Domains organise discovery. Roles and permissions control access.

Common wrong answers.

- `a`: Domains do not restrict access.
- `c`: Domains do not grant access either.
- `d`: Access requests go through workspace roles and item permissions, not domains.
- `b|r2`: Right answer, wrong reason. Every item in the workspace inherits the domain, not only Power BI items.

C9-L1 exercise 2

Hints. (1) Promoted and Certified differ in who applies them. (2) Who names the certifiers? (3) A Fabric admin.

Answer. b) They can request certification. Only users a Fabric admin specifies can certify Because: r1) Any user can request certification, but only users specified by a Fabric administrator can certify.

Promotion is self-service. Certification is an authorised review.

Common wrong answers.

- `a`: Contributors can promote. Certifying needs a name on the Fabric admin's list.
- `c`: Certification is a human review. No test applies it.
- `d`: Any item except a Power BI dashboard can be certified.
- `b|r2`: Right answer, wrong reason. The workspace Admin role does not grant certification rights. The Fabric admin's list does.

C9-L1 exercise 3

Hints. (1) Tenant-level role first. (2) The domain role for workspace admins. (3) `admin` and `contributor`.

Answer.

```
Creates domains and names domain admins: Fabric admin
Assigns their own workspace to a domain: domain contributor, who must be a workspace admin
```

Fabric admins create domains. Domain contributors, who are workspace admins, assign their own workspaces.

Common wrong answers.

- g2 = `admin`: Domain admins associate workspaces too, but the role for workspace admins assigning their own workspaces is domain contributor.

C9-L1 exercise 4

Hints. (1) Handle the dashboard first. (2) Start from promoted and certified. (3) Add master_data only for data item types.

Answer.

```python
DATA_ITEM_TYPES = frozenset({"lakehouse", "warehouse", "semantic_model", "kql_database"})
NO_BADGE_TYPES = frozenset({"dashboard"})


def allowed_badges(item_type: str) -> set[str]:
    """Endorsement badges an item type can receive.

    Args:
        item_type: Fabric item type, such as "lakehouse" or "report".

    Returns:
        Allowed badge names.
    """
    if item_type in NO_BADGE_TYPES:
        return set()
    badges = {"promoted", "certified"}
    if item_type in DATA_ITEM_TYPES:
        badges.add("master_data")
    return badges
```

Encode the rules as constants and small branches, so a rule change touches one line.

C9-L1 exercise 5

Hints. (1) Which feature gives catalog filtering? (2) Which badge marks the single source of truth? (3) What controls access?

Answer.

- A domain gives filtering in the OneLake catalog and a home for delegated settings.
- One master data copy, with shortcuts for other uses (lesson C3-L1).
- Access comes from roles, never from domains.

Domains for discovery, master data for the authoritative copy, roles for access.

Common wrong answers.

- domain = `naming`: A prefix helps a little. A domain adds catalog filtering, delegated settings and named owners.
- customer = `promote_all`: Three promoted copies tell users nothing about which one to trust.
- restrict = `by_domain`: Domain assignment does not affect access.

C9-L2 OneLake security, roles down to rows and columns

C9-L2 exercise 1

Hints. (1) Within a role: intersection. Across roles? (2) Union. (3) Row filters combine with OR.

Answer. a) North and South rows Because: r1) Roles combine as a union. Row filters from different roles combine with OR.

Across roles, access adds up. Plan roles so their union is what you intend.

Common wrong answers.

- `b`: Across roles, filters combine with OR, not AND.
- `c`: Role order does not matter. Access is the union.
- `d`: Only rows allowed by at least one role.
- `a|r2`: Right answer, wrong reason. OneLake unions the roles. It does not pick one.

C9-L2 exercise 2

Hints. (1) Which users does OneLake security govern? (2) Viewers and Read-permission users. (3) Contributors have write access already.

Answer. b) All rows: Contributors already have data access through their workspace role Because: r1) OneLake security governs Viewers and users

with Read permission. Admins, Members and Contributors already have data access.

Restrict readers by making them Viewers first.

Common wrong answers.

- `a`: Contributors are not restricted by OneLake security roles.
- `c`: The workspace role gives them full data access.
- `d`: The read succeeds with all rows.
- `b|r2`: Right answer, wrong reason. Spark enforces OneLake security (GA). The Contributor role is why it does not apply here.

C9-L2 exercise 3

Hints. (1) The engines behave differently. (2) Which one is stricter? (3) The SQL endpoint.

Answer. a) Spark returns the allowed columns. The endpoint returns an error Because: r1) Microsoft Learn documents that CLS behaves differently: Spark shows only allowed columns, the endpoint blocks the query.

List columns explicitly in SQL queries that users with CLS will run.

Common wrong answers.

- `b`: The SQL analytics endpoint blocks queries that touch hidden columns.
- `c`: Spark succeeds and omits the hidden column.
- `d`: Hidden columns are removed or blocked, not nulled.
- `a|r2`: Right answer, wrong reason. Spark enforces CLS by dropping the hidden column.

C9-L2 exercise 4

Hints. (1) Collect the column sets from roles that include the table. (2) No grants: return an empty set. (3) `set().union(*grants)` for Spark, `set.intersection(*grants)` for the endpoint.

Answer.

```
1 from typing import Literal
2
3
4 def visible_columns(
5     roles: list[dict[str, set[str]]],
6     table: str,
7     engine: Literal["spark", "sql_endpoint"],
8 ) -> set[str]:
9     """Columns a user sees in one table across their OneLake security
    roles.
```

```
10
11      Args:
12          roles: Each role maps table names to the columns it allows.
13          table: Table being queried.
14          engine: "spark" (union across roles) or
        "sql_endpoint" (intersection).
15
16      Returns:
17          Visible column names. Empty if no role includes the table.
18
19      Raises:
20          ValueError: If the engine is not recognised.
21      """
22      if engine not in ("spark", "sql_endpoint"):
23          raise ValueError(f"unknown engine: {engine}")
24      grants = [role[table] for role in roles if table in role]
25      if not grants:
26          return set()
27      if engine == "spark":
28          return set().union(*grants)
29      return set.intersection(*grants)
```

The same roles give different column sets in different engines. Test the engine your users query.

C9-L2 exercise 5

Hints. (1) Who does OneLake security govern? (2) Where do related row and column rules go? (3) Which Direct Lake mode enforces OneLake security?

Answer.

- Viewers are governed by OneLake security. Contributors are not.
- Row and column rules for the same table belong in one role. Split roles with different columns and a row filter are rejected.
- Direct Lake on OneLake enforces OneLake security (GA).
- DefaultReader grants full reads, and roles combine by union, so it overrides your restrictions.

Viewers, one role per audience, a supported engine mode, and no leftover default grants.

Common wrong answers.

- ws_role = `contributor`: Contributors already have data access. OneLake roles do not restrict them.
- roles = `split`: Two roles with different columns where one filters rows is a combination OneLake does not support. Keep them in one role.

- model = `import`: Direct Lake on OneLake is on the list of engines that enforce OneLake security. An import model holds its own copy and needs its own security rules.
- default = `keep`: Roles combine as a union. DefaultReader's full access wins over your filters.

C9-L3 Sensitivity labels, Purview and lineage

C9-L3 exercise 1

Hints. (1) Was the report's label set by hand? (2) Can inheritance downgrade? (3) No to both.

Answer. b) It stays Highly Confidential Because: r1) Downstream inheritance never overwrites a manually applied label, and never downgrades a label.

Inheritance only raises or fills in labels. It never lowers them or overrides a person.

Common wrong answers.

- `a`: Inheritance never replaces a label with a less restrictive one, and never touches a manual label.
- `c`: The report's label was set by hand, so inheritance leaves it alone.
- `d`: Inheritance does not remove labels.
- `b|r2`: Right answer, wrong reason. Reports do inherit labels from their semantic model. This one was set manually.

C9-L3 exercise 2

Hints. (1) Which feature follows dependencies? (2) It starts from lineage. (3) Impact analysis.

Answer. a) Impact analysis Because: r1) Impact analysis shows affected downstream items and workspaces and can notify their contacts.

Lineage shows the graph. Impact analysis reads it from one item downwards.

Common wrong answers.

- `b`: Domains group items for discovery. They do not track dependencies.
- `c`: Labels classify sensitivity, not dependencies.
- `d`: The Metrics app shows capacity use (lesson C10-L3).
- `a|r2`: Right answer, wrong reason. Impact analysis informs and notifies. It does not block changes.

C9-L3 exercise 3

Hints. (1) Named users keep what they already have. (2) Everyone else? (3) `keeps` and `blocked`.

Answer.

```
Item labelled Highly Confidential, policy allows the HR group.
HR group member with Viewer access: keeps their Viewer access.
Finance user with Viewer access: blocked.
```

A protection policy never adds permissions. It removes access from everyone it does not name.

Common wrong answers.

- g2 = `keeps`: Users not named in the policy are blocked, even with a workspace role.

C9-L3 exercise 4

Hints. (1) Validate both labels against order first. (2) Unlabelled takes upstream. Manual keeps current. (3) Compare `order.index` values.

Answer.

```python
def inherit_label(
    current: str | None, manual: bool, upstream: str, order: list[str]
) -> str | None:
    """Downstream label after the upstream label changes.

    Args:
        current: The downstream item's label, or None.
        manual: Whether a person applied the current label.
        upstream: The upstream item's new label.
        order: Labels from least to most restrictive.

    Returns:
        The downstream item's label after inheritance.

    Raises:
        ValueError: If a label is not in order.
    """
    for label in (current, upstream):
        if label is not None and label not in order:
            raise ValueError(f"unknown label: {label}")
    if current is None:
        return upstream
    if manual:
        return current
    if order.index(upstream) > order.index(current):
        return upstream
    return current
```

Encoding the rule makes its asymmetry visible: labels move up by inheritance, never down.

C9-L3 exercise 5

Hints. (1) What travels with an Excel export? (2) What blocks a person with a link? (3) What lists downstream items?

Answer.

- A label on the model flows to its reports and stays on Excel exports.
- A protection policy blocks anyone outside the HR group, whatever they were sent.
- Impact analysis names what breaks before it breaks.

Label, protect, and check impact before change.

Common wrong answers.

- label = `unlabelled`: Workspace restrictions stop at the workspace. Labels travel with exports.
- policy = `trust`: A protection policy blocks access even when a link reaches the wrong person.
- change = `rename`: Fixing after the break costs a morning of errors across workspaces. Check first.

C9-L4 Audit evidence, Purview scans, label coverage and the audit log

C9-L4 exercise 1

Hints. (1) The question is about people's actions. (2) Which source records activities? (3) The audit log.

Answer. a) The unified audit log in Microsoft Purview

Fabric records user activities in the Purview unified audit log.

Common wrong answers.

- `b`: A scan lists items and lineage. It records no user activity.
- `c`: Coverage shows which items carry labels, not who opened them.
- `d`: Lineage shows how items connect, not who used them.

C9-L4 exercise 2

Hints. (1) How long does Audit (Standard) keep records? (2) Is that longer or shorter than the period? (3) Export before records expire.

Answer. a) Export audit records on a schedule into storage you control Because: r1) Audit (Standard) keeps records for 180 days, so the first months would expire before the end.

Records older than 180 days expire under Audit (Standard), so export on a schedule.

Common wrong answers.

- `b`: By then, records older than 180 days have expired.
- `c`: Audit (Standard) keeps 180 days. A year needs Premium or your own exports.
- `d`: A scan records items, not who accessed them.
- `a|r2`: Right action, wrong reason. Seven days is only the default search range. Retention is the limit.

C9-L4 exercise 3

Hints. (1) Inventory comes from scanning. (2) Protection is measured as a share of items labelled. (3) `scan`, `coverage`, `audit`.

Answer.

```
1 What do you hold?          a Purview Data Map scan of the Fabric tenant
2 How is it protected?       label coverage in the OneLake catalog Govern
    tab
3 Who touched it?            the unified audit log
```

A scan for inventory, label coverage for protection, the audit log for access.

Common wrong answers.

- g1 = `label`: The scan lists items and lineage. It does not label Fabric data.

C9-L4 exercise 4

Hints. (1) Keep a [labelled, total] pair per workspace. (2) Check `"workspace" in item` before you use it. (3) Divide only for workspaces that have items, so an empty list returns `{}`.

Answer.

```python
def label_coverage(items: list[dict]) -> dict[str, float]:
    """Share of items with a sensitivity label, per workspace.

    Args:
        items: Dicts with "workspace" and "label", where label is a name
    or None.

    Returns:
        Workspace to the share of its items that carry a label, from 0.0
    to 1.0.
```

```
    Raises:
        ValueError: If an item has no "workspace" key.
    """
    counts: dict[str, list[int]] = {}
    for item in items:
        if "workspace" not in item:
            raise ValueError(f"item without a workspace: {item}")
        tally = counts.setdefault(item["workspace"], [0, 0])
        tally[1] += 1
        if item.get("label") is not None:
            tally[0] += 1
    return {workspace: labelled / total for workspace, (labelled, total)
in counts.items()}
```

Count per workspace, so the gaps show where to act. A workspace exists only once it has items, so no division is by zero.

C9-L4 exercise 5

Hints. (1) Which option gives a date and a count? (2) How long do audit records last? (3) Whose permissions apply after the scan?

Answer.

- A scheduled, scoped scan gives a dated inventory an auditor can repeat.
- Monthly exports keep records past the 180-day retention of Audit (Standard).
- Purview permissions, not Fabric roles, govern scanned metadata, so restrict the collection.

Scheduled scans, monthly audit exports and a restricted Purview collection make evidence you can show.

Common wrong answers.

- inventory = `list`: A manual list is a statement, not system evidence.
- access = `later`: Records older than 180 days will have expired.
- metadata = `inherit`: Once in Purview, metadata follows Purview permissions. Fabric roles do not carry over.

C9-L5 Attribute-based access, rows by business unit

C9-L5 exercise 1

Hints. (1) Think about what the filter would need to know. (2) Can a OneLake row filter refer to the current user? (3) Static filters only.

Answer. a) OneLake RLS roles don't support dynamic queries, so a filter cannot look up who is asking

OneLake RLS rules are static. Bring attributes in through groups or an entitlements table.

Common wrong answers.

- `b`: OneLake security has RLS. Its filters are static values.
- `c`: Roles accept Entra users and groups.
- `d`: The reverse. OneLake security supports only grant roles, with deny by default.

C9-L5 exercise 2

Hints. (1) Where does the group get its members from? (2) Who could change that value? (3) Audit write permissions on the attribute.

Answer. a) Who can write the department attribute, in Entra and in on-premises Active Directory Because: r1) Membership is only as safe as the write controls on the attribute the rule reads.

A user who can edit the attribute can join the group. Audit attribute write permissions at the source.

Common wrong answers.

- `b`: The name has no effect on who becomes a member.
- `c`: Both produce the same rule. The risk lies in the attribute.
- `d`: If users can edit their own department, they can add themselves to the group.
- `a|r2`: Right check, wrong reason. Dynamic groups are GA. The risk is who can change the attribute.

C9-L5 exercise 3

Hints. (1) The Entra equals operator starts with a hyphen. (2) The T-SQL statement names a security policy. (3) `-eq`, `SECURITY`, `USER_NAME`.

Answer.

```
1 Entra dynamic group rule:     user.department -eq "HR"
2 Warehouse row security:       CREATE SECURITY POLICY ... ADD FILTER
     PREDICATE ...
3 Current user in the predicate: USER_NAME()
```

Entra rules use `-eq`. Warehouse RLS uses `CREATE SECURITY POLICY` with a predicate that reads `USER_NAME()`.

Common wrong answers.

- g2 = `ROW ACCESS`: `CREATE ROW ACCESS POLICY` is Snowflake syntax. Fabric Warehouse uses `CREATE SECURITY POLICY`.

C9-L5 exercise 4

Hints. (1) Look up the user's units once, with an empty list as the default. (2) Lower-case the user before the lookup. (3) Use `row.get("business_unit")` so a row without the key is skipped.

Answer.

```python
def allowed_rows(
    rows: list[dict], entitlements: dict[str, list[str]], user: str
) -> list[dict]:
    """Rows the user may see, from an entitlements map.

    Args:
        rows: Dicts with a "business_unit" key.
        entitlements: Lower-case user principal name to allowed business units.
        user: The user principal name of the person asking.

    Returns:
        The allowed rows in their original order. Empty for a user with no entry.
    """
    units = set(entitlements.get(user.lower(), []))
    return [row for row in rows if row.get("business_unit") in units]
```

Deny by default: no entitlement, no rows. Lower-case the user principal name, and never let a row without a unit through.

C9-L5 exercise 5

Hints. (1) Which option survives a reorganisation with a data change? (2) What does deny by default mean for a missing entry? (3) The policy is only as safe as the table it reads.

Answer.

- Keep who sees what as data in an entitlements table, read by one security policy.
- No entitlement means no rows. Deny by default.
- Restrict and audit writes to the table, because the policy trusts it.

One policy over a stewarded entitlements table, deny by default, with audited writes.

Common wrong answers.

- source = `code`: Every move becomes a code change, and personal data sits in code.
- unknown = `all`: A missing entry would expose every unit. Deny by default.
- writes = `anyone`: Anyone who can edit the table can grant themselves access.